

Cluster Maintenance, Backup & Restore

Cordoning, draining, certificate rotation, backup, restore, and disaster recovery - real operations, real downtime measured in seconds.

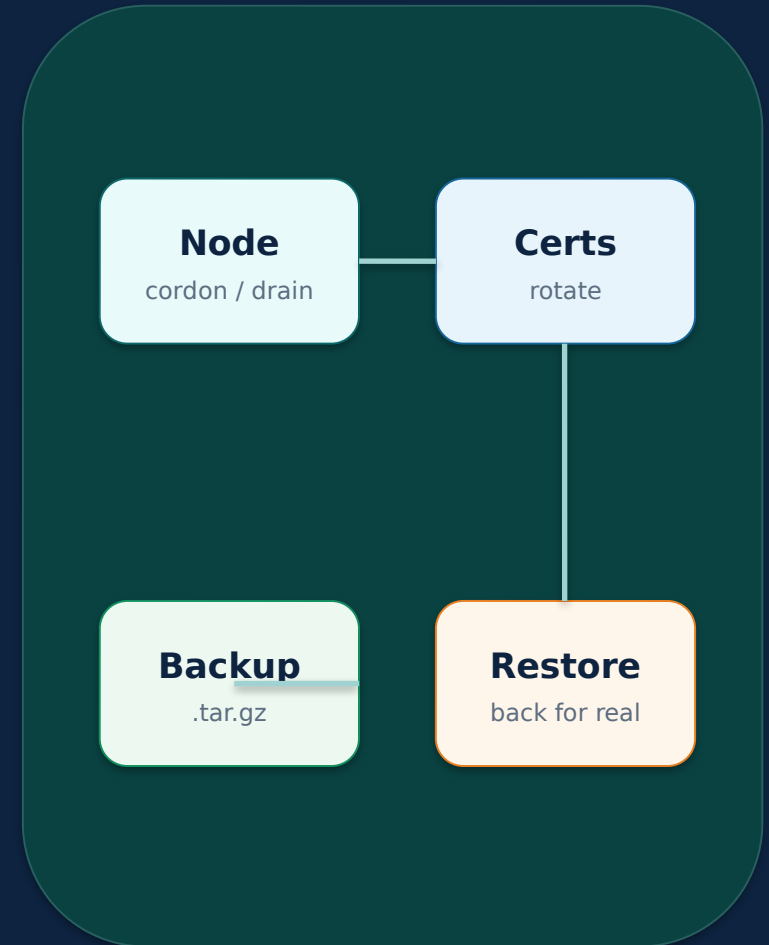
CORDON

DRAIN

CERTS

BACKUP

RESTORE



Where this sits in the course

Day 01-03

Workloads, Services, Secrets, RBAC, networking, Ingress.

Day 04

Storage and resource governance
- everything today proves survives.

Today

Keeping the CLUSTER ITSELF
alive -
maintenance and recovery.

The question today answers

Every other day changed objects INSIDE the cluster. What happens when you need to change (or fix) the cluster itself?

```
kubectl get nodes
```

```
sudo k3s certificate check
```

What this day is really about

Every other day changed objects inside the cluster. This day changes the cluster's **OWN** control plane and data - the scope was agreed explicitly before building it.

Operation	Executed for real?
Cordon / uncordon	Yes - zero disruption risk
Drain	--dry-run=client only - nowhere to reschedule to
Certificate rotation	Yes - ~2s downtime, old certs auto-backed-up
Backup	Yes - real stop/tar/start of the live datastore
Restore	Yes - a real namespace deleted and restored
k3s version upgrade	NOT executed - one-way door on the shared cluster

What's verified about this cluster

```
$ k3s --version
k3s version v1.36.4+k3s1

$ sudo ls /var/lib/rancher/k3s/server/db
state.db  state.db-shm  state.db-wal  # SQLite - not etcd

$ sudo k3s certificate check | head -2
client-kube-apiserver.crt: ... is ok, expires at 2027-09-08...
```

A single k3s server defaults to embedded SQLite ("kine"), not etcd - that one fact shapes this day's entire backup/restore approach.

Node cordoning

```
$ kubectl cordon lab-g2-vm2
$ kubectl get node lab-g2-vm2
NAME          STATUS
lab-g2-vm2    Ready,SchedulingDisabled

$ kubectl apply -f ex01-node-cordoning/test-pod.yaml
$ kubectl describe pod cordon-test | tail -1
Warning FailedScheduling ... 1 node(s) were unschedulable.

$ kubectl -n kube-system get pods | grep traefik
traefik-...  1/1  Running      # untouched
```

SchedulingDisabled is independent of Ready. Uncordoning is instant - the waiting Pod schedules immediately.

Node drain: what it actually needs

```
$ kubectl drain lab-g2-vm2 --dry-run=client
error: ... cannot delete Pods with local storage
      (use --delete-emptydir-data), cannot delete
      DaemonSet-managed Pods (use --ignore-daemonsets)

$ kubectl drain lab-g2-vm2 --ignore-daemonsets \
  --delete-emptydir-data --dry-run=client
evicting pod kube-system/coredns-... (dry run)
evicting pod kube-system/traefik-... (dry run)
node/lab-g2-vm2 drained (dry run)
```

drain refuses by default: emptyDir data would be lost, DaemonSet Pods would just get recreated on the same node anyway.

Why a real drain wasn't executed here

Look at that eviction list again: CoreDNS, the CSI controller, local-path-provisioner, Traefik, metrics-server - essentially every system component every other day depends on.

Real multi-node cluster: drain moves that workload elsewhere, the cluster keeps functioning throughout.

This one-node cluster: a real drain leaves all of it Pending until uncordon - DNS, ingress, storage, metrics down.

--dry-run=client gives the exact same planning information with zero actual disruption - the whole value of rehearsing a drain, on any cluster.

Certificate rotation: executed for real

```
$ ... serving-kube-apiserver.crt | openssl x509 -serial -dates # BEFORE
serial=43B6430A80F15C47

$ sudo k3s certificate rotate
msg="Successfully backed up certificates to .../tls-1789321189"

$ sudo systemctl restart k3s
$ kubectl get nodes # polled every 2s
API back after ~2s

$ ... serving-kube-apiserver.crt | openssl x509 -serial -dates # AFTER
serial=05B85C75E3372D65 # different
```

rotate backs up the old certs automatically. ~2s of real downtime; every workload came back Running unassisted.

Why this session's own kubectl never noticed

This terminal's ~/.kube/config still has the ORIGINAL client cert (copied once, at install time).

k3s certificate rotate reissues LEAF certificates only - it does not touch the CA (rotate-ca is separate, far more disruptive).

The old client cert is still validly signed by the same, unchanged CA - the API server keeps trusting it.

**Rotation invalidates nothing already issued - it only changes what gets issued NEXT.
That's why it's safe to do routinely, not just before expiry.**

Backup: SQLite, not etcd

```
$ sudo k3s etcd-snapshot save  
level=fatal msg="...etcd datastore disabled"
```

```
$ bash ex04-backup/backup-datastore.sh /root/k3s-backups  
Stopping k3s...  
Archiving /var/lib/rancher/k3s/server ->  
.../k3s-server-20260913-231134.tar.gz  
Starting k3s...
```

API back after ~2s

etcd-snapshot only works with embedded etcd (--cluster-init). A single server's default SQLite datastore needs a plain file-level backup of db/, tls/, and token instead - stopped first so the copy is consistent.

Restore: delete it for real, get it back for real

```
$ kubectl delete namespace canary # simulate loss
$ kubectl get namespace canary
Error ... namespaces "canary" not found

$ bash ex05-restore/restore-datastore.sh <backup>.tar.gz
Moving current state aside (not deleting) -> ...
API back after ~2s

$ kubectl get namespace canary
NAME      STATUS   AGE
canary    Active   2m12s # ORIGINAL timestamp
```

AGE 2m12s is the tell: a true point-in-time restore, not a recreation. The pre-restore state is moved aside, never deleted outright.

RTO / RPO: measured, not estimated

Metric	What it means	This cluster, measured
RTO	how long you're down	~2 seconds - the stop -> copy -> start window
RPO	how much you can lose	the age of your most recent backup

Every number above came from an actual date-stamped run in this course's OUTPUT.md files - not a vendor claim or a guess.

Disaster recovery: what needs a human

Event	Recovers on its own?	What brings it back
A Pod crashes	Yes	its controller (Day 01/07's self-healing loop)
The node reboots	Mostly	nothing, if disks are intact
Datastore corrupted/lost	No	ex05's restore, from an ex04 backup taken before the loss
The whole VM/disk destroyed	No	a backup copied OFF this VM - the gap on Slide 14

The gap this course can't close

ex04's .tar.gz sits on the SAME DISK it's backing up.

Survives

- a bad `rm -rf` of the wrong namespace/directory
- a failed upgrade
- a corrupted datastore file

Does NOT survive

- the disk itself failing
- the whole VM being destroyed

Closing this means copying the backup to a different host/object storage - routine in a real deployment, but there's no second location in this single-VM course to demonstrate it against. Checklist item, not a lab - same honesty applied to every single-node limit since Day 03.

Upgrade strategy: the real procedure (rehearsed)

```
$ k3s --version
k3s version v1.36.4+k3s1
$ curl -s .../releases/latest | grep tag_name
      "tag_name": "v1.36.4+k3s1"    # already latest stable
```

```
kubectl cordon lab-g2-vm2
curl -sL https://get.k3s.io | \
  INSTALL_K3S_VERSION=vX.Y.Z+k3s1 sh -s - server ...
kubectl get nodes      # confirm the new VERSION
kubectl get pods -A     # confirm recovery
                        kubectl uncordon lab-g2-vm2
```

Textbook-safe: cordon -> drain -> upgrade -> uncordon, one node at a time. With one node, it collapses to cordon -> upgrade (short real restart) -> uncordon.

Upgrade rollback = restore, not downgrade

k3s does NOT officially support downgrading a server's version - the datastore can move forward in ways that aren't guaranteed reversible.

The supported rollback path if an upgrade goes wrong: ex05's restore - a backup taken IMMEDIATELY BEFORE the upgrade, restored onto a reinstalled old binary.

This is why the pre-upgrade checklist leads with "take a backup" - the backup, not the version number, is the actual safety net.

Best-practice checklist

Always cordon before touching a node - even when you're also going to drain it for real.

Treat a `--dry-run=client` drain as mandatory rehearsal on ANY cluster, not just a single-node one.

Rotate certificates on a schedule, not just before expiry - it's cheap and k3s backs up the old ones.

Back up on a schedule, prune old backups, and copy at least one off-node - the copy step is what most setups skip.

Practice the restore command before you need it for real.

Never upgrade without a fresh backup taken immediately before - treat them as one decision.

Hands-on labs — built and verified

Lab	Proves
ex01 node cordoning	new Pods blocked, existing Pods untouched, instant recovery
ex02 node drain	real eviction plan via --dry-run=client, zero disruption
ex03 certificate rotation	new serials, ~2s downtime, old certs auto-backed-up
ex04 backup	real stop/tar/start of the SQLite datastore
ex05 restore	a deleted namespace restored byte-for-byte
ex06 disaster recovery	runbook: self-heals vs needs a human, measured RTO/RPO

Same pattern as Days 01-07: manifests/scripts + LAB-MANUAL.md (steps + troubleshooting) + OUTPUT.md (real captured output).

RECAP

The cluster is data too. Back it up like it matters.

Cordon first, always - it's free and instant.

A --dry-run drain tells you the truth about what depends on a node before you evict anything for real.

Certificate rotation replaces leaf certs only - already-issued clients keep working through it.

This datastore is SQLite, not etcd - backup is a file copy, and etcd-snapshot says so instead of failing silently.

RTO and RPO aren't abstractions - this day measured both, in seconds, on a real backup and a real restore.

A version upgrade is a one-way door; the backup taken right before it is the actual rollback plan.